# RAG Pipeline Report: Baseline Retrieval versus Cross-Encoder Re-Ranking

Emil Joji 2025201040

**Corpus:** The Laws of Cricket (2017 Code, 4th Edition, 2026) | **Bi-encoder:** all-MiniLM-L6-v2 | **Cross-encoder:** ms-marco-MiniLM-L-6-v2 | **Generator:** Qwen3-14B (4-bit)

Code Repository: https://github.com/algostride/RAG-Baseline-vs-ReRanking

## 1 Setup & Environment

This report compares two retrieval pipelines run over the same chunked corpus: a single-stage bi-encoder baseline, and a two-stage pipeline that reranks the baseline's candidates with a cross-encoder before generation. Everything else – the chunk corpus, the first-stage embedding model, and the prompt template – is held fixed, so any difference in the final answer comes from how the retrieved passages are ordered, not from a change in the underlying data. Generation runs locally through a 4-bit quantized Qwen3-14B.

The chosen dataset is the Laws of Cricket (42 Laws, 5 appendices, 90 pages) – a regulatory document, as suggested by the brief, selected because its dense web of exceptions, suspensions, and role-specific permissions is exactly where a bi-encoder's context-free embeddings are most likely to retrieve something topically close but substantively wrong.

Text was extracted with `pypdf` (232,012 characters); the back-of-book index was dropped since it just repeats every heading as a page number with no clause text. Chunking follows the document's own structure rather than a fixed sliding window: the text is first split on the 42 Law and 5 Appendix headings, then packed into ∼90-word windows with one sentence of overlap carried forward, with every chunk prefixed by its section heading for context. This produced **48 sections and 758 chunks**. One known wrinkle: the Appendices sit in a multi-column layout in the source PDF and extract slightly out of order; none of the stress tests below rely on them.

## 2 Two-Stage Retrieval Implementation

### 2.1 Pipeline 1 (Baseline Retrieval)

Every chunk is embedded once, offline, with all-MiniLM-L6-v2 (L2-normalized, so cosine similarity is just a dot product). A query is embedded the same way at search time and scored against all 758 chunks by inner product; the top 10 are retrieved, and the top 3 of those are taken directly into the prompt to generate Answer 1.

### 2.2 Pipeline 2 (Advanced Re-Ranking)

Starts from the identical top-10 candidates returned by Pipeline 1, so both pipelines see the same pool. All 10 (query, chunk) pairs are scored jointly by the cross-encoder `ms-marco-MiniLM-L-6-v2`, re-sorted by that score, and the new top 3 are fed into the same prompt structure used by Pipeline 1 to generate Answer 2.

Both pipelines share one system prompt: answer only from the numbered passages given, cite every claim, check each passage individually before concluding an answer is absent, never treat a term in the question as equivalent to a different term in the passages unless the text says so, and say "I don't know" rather than guess. If generation fails for any reason the pipeline falls back to an extractive stub so a result is always returned. As a sanity check before stress-testing, an easy query ("how many players are on each side?") was run through both pipelines: the correct passage was already ranked

#1 by the baseline, so reranking changed nothing and both pipelines gave the identical, correct answer – confirming the mechanics end to end before looking at the harder cases below.

# 3 Failure Mode Analysis (Stress Testing)

Four queries were built directly against the Laws of Cricket text, each paired with a `true_snippet` (the chunk that actually answers it) and a `decoy_snippet` (a chunk that competes for the baseline's attention without fully answering the question):

| ID | Category | Query |
|---|---|---|
| notification_exception | Exception / Corner Case | From what point is an interruption counted toward a fielder's outstanding Penalty time? |
| suspension_exception | Exception / Suspension | Is time spent off the field under a Level 3 suspension treated as ordinary absence for unserved Penalty time? |
| negation_constraint | Negation / Constraint | If a substitute has already fielded for a player, can that player still bowl later in the innings? |
| other_document_specific | Other Document-Specific | Does the ball hitting a batter's leg in front of the stumps give a wicket with no bat contact? |

Table 1: The four stress-test queries and the failure category each targets.

These cover three of the four failure categories the assignment suggests – Exceptions & Corner Cases, Negation & Constraints, and a document-specific failure uncovered in this corpus. The fourth, a keyword trap with high lexical overlap to an irrelevant passage, was attempted twice against this document with this embedding model, but no reproducible case could be constructed; this is reported as a negative finding rather than silently omitted. Rank changes and the resulting answer differences for all four queries are documented in Section 4.2 below.

# 4 Analysis and Report

## 4.1 Architecture Comparison

A bi-encoder (Pipeline 1) embeds the query and each chunk independently, with no knowledge of the other, and relevance is calculated afterward with a cheap vector operation such as cosine similarity or a dot product. Since chunk embeddings don't depend on the query, they can be computed once, offline, and reused for every future query, which makes the bi-encoder efficient for retrieving from a large corpus.

A cross-encoder (Pipeline 2) processes the query and a candidate chunk together as a single input sequence. Its joint self-attention lets query tokens interact directly with chunk tokens, helping it catch subtle relationships such as exceptions, negation, and similar-looking but semantically different passages. Unlike bi-encoder embeddings, its scores can't be precomputed independently of the query: each query-chunk pair has to be scored separately, which adds latency. The two-stage pipeline combines both strengths: the bi-encoder retrieves an initial candidate set efficiently, and the cross-encoder reranks those candidates by relevance to the query.

## 4.2 Qualitative & Rank Shift Analysis

| ID | True chunk (shift) | Decoy chunk (shift) | Answers differ |
|---|---|---|---|
| notification_exception | 360 ($4 \rightarrow 2$) | 359 ($1 \rightarrow 1$) | Yes |
| suspension_exception | 364 ($4 \rightarrow 1$) | 366 ($2 \rightarrow 6$) | Yes |
| negation_constraint | 352 ($6 \rightarrow 2$) | 350 ($1 \rightarrow 6$) | Yes |
| other_document_specific | 502 ($6 \rightarrow 3$) | 533 ($1 \rightarrow 5$) | Yes |

Table 2: Rank shift, baseline $\rightarrow$ reranked, for the four stress-test queries.

**notification_exception.** Chunk 360 climbed from rank 4 to rank 2; decoy chunk 359 held rank 1 both times, so it stayed in the baseline's top 3 (359, 362, 358) while the real answer sat just outside. After reranking the top 3 became (359, 360, 358). Both answers correctly identify that the clock starts when the fielder notifies an umpire, but only the reranked answer adds the actual exception – stoppage time *before* that notification doesn't count. The decoy is topically perfect (same subject, Penalty time) but missing the qualifier; a bi-encoder has no good way to separate "the rule" from "the rule, except when," which is exactly what joint attention is built to catch.

**suspension_exception.** Chunk 364 jumped from rank 4 to rank 1; decoy 366 fell from rank 2 to rank 6, so the correct chunk missed the baseline's top 3 but led the reranked one. Both answers reach the right conclusion – no, provided the fielder returns at the end of the suspension – but the reranked answer is grounded directly in the governing passage, while the baseline arrives there less directly. The reranked answer does add one unsupported aside about "Penalty time served," worth flagging as a spot where reranking fixed retrieval but generation still overreached slightly.

**negation_constraint.** Chunk 352 rose from rank 6 to rank 2 while decoy 350 dropped from rank 1 to rank 6 – the final context shifted from {350, 351, 356} to {351, 352, 356}. Both answers land on the correct "yes," but the reranked one is shorter and doesn't pull in an unrelated Penalty-time restriction the way the baseline's does. The decoy shares the words "substitute" and "bowl" with the query, but it's actually about what the *substitute* can't do, not what the *original player* can do – a distinction a bag-of-meaning embedding tends to blur and joint attention does not.

**other_document_specific (LBW).** The clearest case: chunk 502 moved from rank 6 to rank 3 (decoy 533 fell from rank 1 to rank 5), and it turned an abstention into a correct answer. The baseline's top 3 – Stumped, a related LBW mention, No Ball – all touch "a batter getting out" but none state the actual LBW conditions, so the model correctly said it didn't know given what it had. With chunk 502 in the reranked top 3, it confirmed that LBW can apply without bat contact and listed the real conditions from Law 36. Cricket's dismissal laws share a lot of surface vocabulary (wicket, striker, ball), so lexical overlap alone was never going to find the right one here.

**Summary.** Reranking moved the true chunk into the top 3 in every case (baseline ranks 4, 4, 6, 6 → reranked ranks 2, 1, 2, 3) and changed the generated answer every time: restoring a missing exception in two cases, tightening an already-correct answer in one, and turning an abstention into a grounded answer in the last. Reranking doesn't guarantee a perfect answer on its own, though – the suspension_exception case shows that – it mainly guarantees the model is reasoning from the right passage, not that the generated prose stays perfectly within it.

### 4.3 Latency Profiling

Retrieval-side latency (query embedding plus search for Pipeline 1; the same plus cross-encoder scoring for Pipeline 2) was timed over 5 repetitions per query. LLM generation time is excluded, since it depends entirely on the generation backend and would swamp this comparison.

| Query | Pipeline 1 (ms) | Pipeline 2 (ms) | Overhead (ms) | Overhead (%) |
|---|---|---|---|---|
| Notification exception | 7.77 | 26.21 | 18.43 | 237.2 |
| Suspension exception | 6.30 | 25.85 | 19.54 | 310.0 |
| Negation constraint | 6.56 | 23.98 | 17.42 | 265.5 |
| LBW / document-specific | 6.71 | 31.04 | 24.32 | 362.2 |
| **Mean** | **6.84** | **26.77** | **19.94** | **291.7*** |

Table 3: Retrieval-side latency, baseline vs. baseline plus reranking. *Computed from the unrounded means.

Pipeline 1 averaged 6.84 ms; Pipeline 2 averaged 26.77 ms – reranking adds roughly 20 ms, about 3.9× the baseline. Overhead ranged from 17.4 to 24.3 ms across queries (the LBW query was both the slowest and the most improved by reranking), though four queries and five repetitions is too small

a sample to draw a real relationship between query length and overhead. Either way, the absolute cost stayed under 25 ms added per query – cheap next to what it bought in Section 4.2.

### 4.4 Scaling Question

A cross-encoder's score depends on both the query and the chunk together, so it can't be cached the way a bi-encoder's chunk embeddings can: scoring an entire corpus of $N$ chunks costs $N$ transformer forward passes per query, a cost that grows linearly with corpus size. A bi-encoder's brute-force search is also $O(N)$, but over cheap vector operations against embeddings computed once – and approximate nearest-neighbor indexes such as HNSW cut that further still. At 758 chunks, reranking everything would be manageable; at a million, a full cross-encoder sweep per query would be far slower than vector search, by how much depending on model, hardware, and batching strategy. The standard fix, and the one implemented here, is a two-stage retrieval funnel: let the bi-encoder do the cheap part (retrieve 10 candidates) and the cross-encoder do the accurate part on that short list (rerank to a final top 3). This keeps the expensive joint scoring confined to a small candidate set rather than the full corpus, preserving the scalability of embedding search while applying joint query-passage scoring exactly where it matters.

## 5 Conclusion

Across four stress-test queries spanning exceptions, suspensions, negation, and a document-specific dismissal rule, baseline retrieval left the decisive passage outside the top 3 every time; reranking put it back in every time, and changed the final answer in all four cases. That gain cost about 20 ms of extra retrieval latency per query (3.9× baseline) – affordable specifically because the cross-encoder only ever looks at the 10 candidates the bi-encoder already found, not the full 758-chunk corpus.

**Limitations.** These results come from a single document, a single bi-encoder/cross-encoder pairing, and four hand-built queries, so the rank shifts above should be read as a demonstration of the mechanism rather than a general benchmark of reranking's benefit. The latency numbers are retrieval-side only and averaged over just 5 repetitions per query; and because generation runs on a locally quantized model, some answer-quality differences (e.g. the unsupported aside in suspension_exception) reflect the generator's behavior as much as the retrieved context itself, which is why Answer 1 vs. Answer 2 comparisons above are read qualitatively rather than scored automatically.

## A Pipeline Configuration Reference

| Parameter | Value |
|---|---|
| Chunk target size | 90 words |
| Chunk overlap | 1 sentence |
| Candidate pool size (top-$K$ search) | 10 |
| Final context size (top-$K$ final) | 3 |
| Embedding model | all-MiniLM-L6-v2 (384-dim) |
| Cross-encoder model | ms-marco-MiniLM-L-6-v2 |
| Generation model | Qwen3-14B (4-bit, NF4 quantization) |
| Random seed | 7 |
| Source document pages | 90 |
| Extracted characters | 232,012 |
| Detected sections / total chunks | 48 / 758 |
| Timing repetitions per query | 5 |

Table 4: Configuration values used by both pipelines, taken from the notebook's setup and corpus-preparation cells.